Documentation of QuineMC Cluskey Program

1) Pre requisite to Understand Program

Basic Understanding of

Programming in C++:

- **1D and 2D Dynamic Array**
- **Regrow and Shrink Concept**
- **Functions**

In DLD (Digital Logic & Design):

- **CH # 4 of Book Digital Fundamental by L FLoyd**
- **Quine McCluskey Method Working**
- **Grey Code Relationship & its Arrangement in K-Map and Quine Mc Cluskey Table (Optional)**
- **Similarities between K-map and Quine MC Cluskey Method (Optional)**

A second semester student of UCP is eligible to make a program.

Notes:

The Quine Mc Cluskey Method consists of two steps.

1) The step 1 is to make pairs to eliminate variables.
2) The step 2 is to separate essential implicants from non essential primte implicants.

The step 1 needs the following no. of steps to be performed.

- Make a table which contains all Minterms or Truth Table. (See table 4-10)

TABLE 4–9

ABCD	*X*	Minterm
0000	0	
0001	1	m_1
0010	0	
0011	1	m_3
0100	1	m_4
0101	1	m_5
0110	0	
0111	0	
1000	0	
1001	0	
1010	1	m_{10}
1011	0	
1100	1	m_{12}
1101	1	m_{13}
1110	0	
1111	1	m_{15}

Abdul Rafy

1 (This step is not taken in Program, however we are not comparing terms within group)Arrange the minterms into groups such that each group should have equal no. of 0s and 1s. (See table 4-10)

TABLE 4–10

Groups	Minterm	TABLE
1	m_1	0001
	m_4	0100
2	m_3	0011
	m_5	0101
	m_{10}	1010
	m_{12}	1100
3	m_{13}	1101
4	m_{15}	1111

2 There will be no comparison in between the minterms within the group to make pairs. The comparison will be between the minterms belonging to different groups to make pairs to eliminate a variable. ***Moreover, comparison should be made or pairing can be made between minterms of adjacent groups.*** (See table 4-11)

3 Check/Mark(✓) all the Minterms which have taken part in any pairing. (See table 4-11)

4 Those terms which have not marked(✓) or not taken part in any pairing will be our prime implicants. (See table 4-11)

5 The Next level table will be made which will contain all the binary of paired terms with elimination of one varible denoted by 'X'. (See table 4-11)

TABLE 4–11

Groups	Minterm	TABLE	NEXT LEVEL TABLE	
1	m_1 ✓	0001	00x1	(m_1, m_3)
	m_4 ✓	0100	0x01	(m_1, m_5)
2	m_3 ✓	0011	010x	(m_4, m_5)
	m_5 ✓	0101	x100	(m_4, m_{12})
	m_{10}	1010	x101	(m_5, m_{13})
	m_{12} ✓	1100	110x	(m_{12}, m_{13})
3	m_{13} ✓	1101	11x1	(m_{13}, m_{15})
4	m_{15} ✓	1111		

Note: We will continue repeating these no. of steps(from 1 to 5) till we reach a stage where we have no term in Next level Table and there is no term which is checked/marked(✓).

Program Tip: If we have no term in Next level Table, meaning if its column length is zero, i.e no term is checked so, no more iteration needed.

<u>Before starting next iteration</u>:

Abdul Rafy

- After each repetition, the 'Next level (binary) Table' will become the binary 'Table' if the 'Next level (binary) Table' binary combinations. To do so we will delete elements from 'Table' column and write the Next Level Table element in it. (See Table 4-12 and 4-13)
- The element in Next Level table then will be removed to enter the result of pairing of terms in new iteration.(See Table 4-14)

Each repetition removes a variable from a minterm taken part in pairing. Forexample, if there is 2 repetition only, the last 'Table' will have binary combinations with 1 X's i.e. with elimination of one variable.

Program Tip: While making the program, we ususallly deal with indices 0,1,2,3,4 which has corresponding minterms m1, m3, m4, m5… . So we don't care about minterms. Our mostly working is with binary of these mineterms which is also on same indexes as that of minterms.

Following example is showing how to achieve step 1.

Forexample, Suppose Input in program is m1, m4, m3, m5, m10, m12,m13, m15.(This example input is already arranged for first step, but it is not necessary to input in any specific order. Just give any input in any order to the program.

Remember: We have nothing to do with minterms except taken as input to get binary and we are dealing with indexes in which order these minterms are saved. You may say each index is representing a unique minterm. Our manipulation is mostly with binary which are also stored at same index in which minterms are stored.

The step 1 needs the following no. of steps to be performed.

- Make a table which contains all Minterms or Truth Table. (See table 4-10)

1st Loop

- (This step is not taken in Program, however we will not be comparing terms within group in the program)Arrange the minterms into groups such that each group should have equal no. of 0s and 1s. (See table 4-10)

Abdul Rafy

TABLE 4–10

Groups	Minterm	Index	TABLE
1	m_1	0	0001
	m_4	1	0100
2	m_3	2	0011
	m_5	3	0101
	m_{10}	4	1010
	m_{12}	5	1100
3	m_{13}	6	1101
4	m_{15}	7	1111

- There will be no comparison in between the minterms within the group to make pairs.
- The comparison will be between the minterms belonging to different groups to make pairs to eliminate a variable. ***Moreover, comparison should be made or pairing can be made between minterms of adjacent groups.*** (See table 4-11)
- Check/Mark(✓) all the Minterms which have taken part in any pairing. (See table 4-11)
- Those terms which have not marked(✓) or not taken part in any pairing will be our **prime implicants**. (See table 4-11)
- The Next level table will be made which will contain all the binary of paired terms with elimination of one varible denoted by 'X'. (See table 4-11)

Prime Implicants	
0	1010
1	
2	

TABLE 4–11

Groups	Minterm		TABLE		NEXT LEVEL TABLE		
1	m_1	✓	0001	0	00x1	0	(m_1, m_3)
	m_4	✓	0100	1	0x01	1	(m_1, m_5)
2	m_3	✓	0011	2	010x	2	(m_4, m_5)
	m_5	✓	0101	3	x100	3	(m_4, m_{12})
	m_{10}		1010	4	x101	4	(m_5, m_{13})
	m_{12}	✓	1100	5	110x	5	(m_{12}, m_{13})
3	m_{13}	✓	1101	6	11x1	6	(m_{13}, m_{15})
4	m_{15}	✓	1111	7			

Checking Duplicates = No Duplicates

- As the Next Level Table has terms or some indexes are checked, so we will **start** next iteration with some manipulation before it.

<u>Before starting next iteration</u>:

- After each repetition, the 'Next level (binary) Table' will become the binary 'Table' if the 'Next level (binary) Table' binary combinations. To do so we will delete elements from 'Table' column and write the Next Level Table element in it.

Abdul Rafy

TABLE 4–12			
Term	TABLE	NEXT LEVEL TABLE	
		00x1 0	(m_1, m_3)
		0x01 1	(m_1, m_5)
		010x 2	(m_4, m_5)
		x100 3	(m_4, m_{12})
		x101 4	(m_5, m_{13})
		110x 5	(m_{12}, m_{13})
		11x1 6	(m_{13}, m_{15})

(a)

TABLE 4–13			
Term	Index	TABLE	NEXT LEVEL TABLE
(m_1, m_3)	0	00x1	00x1 0
(m_1, m_5)	1	0x01	0x01 1
(m_4, m_5)	2	010x	010x 2
(m_4, m_{12})	3	x100	x100 3
(m_5, m_{13})	4	x101	x101 4
(m_{12}, m_{13})	5	110x	110x 5
(m_{13}, m_{15})	6	11x1	11x1 6

(b)

- The element in Next Level table then will be removed to enter the result of pairing of terms in new iteration.(See following 4-11)

TABLE 4–11			
Term	Index	TABLE	NEXT LEVEL TABLE
(m_1, m_3)	0	00x1	
(m_1, m_5)	1	0x01	
(m_4, m_5)	2	010x	
(m_4, m_{12})	3	x100	
(m_5, m_{13})	4	x101	
(m_{12}, m_{13})	5	110x	
(m_{13}, m_{15})	6	11x1	

2nd Loop

- (This step is not taken in Program, however we are not comparing terms within group)Arrange the minterms into groups such that each group should have equal no. of 0s and 1s. (See table 4-10). (See following 4-11)

TABLE 4–11				
Groups	Term	Index	TABLE	NEXT LEVEL TABLE
1	(m_1, m_3)	0	00x1	
	(m_1, m_5)	1	0x01	
	(m_4, m_5)	2	010x	
	(m_4, m_{12})	3	x100	
2	(m_5, m_{13})	4	x101	
	(m_{12}, m_{13})	5	110x	
3	(m_{13}, m_{15})	6	11x1	

- There will be no comparison in between the minterms within the group to make pairs.
- The comparison will be between the minterms belonging to different groups to make pairs to eliminate a variable. ***Moreover, comparison should be made or pairing can be made between minterms of adjacent groups.*** (See table 4-11)
- Check/Mark(✓) all the Minterms which have taken part in any pairing. (See table 4-11)
- Those terms which have not marked(✓) or not taken part in any pairing will be our **prime implicants**. (See table 4-11)
- The Next level table will be made which will contain all the binary of paired terms with elimination of one varible denoted by 'X'. (See table 4-11)

Abdul Rafy

	Prime Implicants
0	1010
1	00x1
2	0x01
3	11x1

TABLE 4–11

Groups	Term	Index	TABLE	NEXT LEVEL Index	TABLE
1	(m_1, m_3)	0	00x1	0	x10x $(m_4, m_5, m_{12}, m_{13})$
	(m_1, m_5)	1	0x01	1	x10x $(m_4, m_5, m_{12}, m_{13})$
	(m_4, m_5)	2 ✓	010x		
	(m_4, m_{12})	3 ✓	x100		
	(m_5, m_{13})	4 ✓	x101		
2	(m_{12}, m_{13})	5 ✓	110x		
3	(m_{13}, m_{15})	6	11x1		

Checking Duplicates= No Duplicates

- As the Next Level Table has terms or some indexes are checked, so we will **start** next iteration with some manipulation before it.

<u>Before starting next iteration</u>:

- After each repetition, the 'Next level (binary) Table' will become the binary 'Table' if the 'Next level (binary) Table' binary combinations. To do so we will delete elements from 'Table' column and write the Next Level Table element in it.

TABLE

Term	Index	TABLE	NEXT LEVEL Index	TABLE
			0	x10x $(m_4, m_5, m_{12}, m_{13})$
			1	x10x $(m_4, m_5, m_{12}, m_{13})$

(a)

TABLE

Term	Index	TABLE	NEXT LEVEL Index	TABLE
$(m_4, m_5, m_{12}, m_{13})$	0	x10x	0	x10x $(m_4, m_5, m_{12}, m_{13})$
$(m_4, m_5, m_{12}, m_{13})$	1	x10x	1	x10x $(m_4, m_5, m_{12}, m_{13})$

(b)

- The element in Next Level table then will be removed to enter the result of pairing of terms in new iteration.(See following 4-11)

TABLE

Term	Index	TABLE	NEXT LEVEL Index	TABLE
$(m_4, m_5, m_{12}, m_{13})$	0	x10x		
$(m_4, m_5, m_{12}, m_{13})$	1	x10x		

3rd Loop

- taken Arrange the minterms into groups such that each group should have equal no. of 0s and 1s. (See table 4-10). (See following 4-11)

TABLE

Groups	Term	Index	TABLE	NEXT LEVEL Index	TABLE
1	$(m_4, m_5, m_{12}, m_{13})$	0	x10x		
	$(m_4, m_5, m_{12}, m_{13})$	1	x10x		

- There will be no comparison in between the minterms within the group to make pairs.
- The comparison will be between the minterms belonging to different (specifically adjacent) groups to make pairs to eliminate a variable. ***Moreover, comparison should be made or pairing can be made only between minterms of adjacent groups.*** (See table 4-11)
- Check/Mark(✓) all the Minterms which have taken part in any pairing. (See table 4-11)
- Those terms which have not marked(✓) or not taken part in any pairing will be our **prime implicants**. (See table 4-11)

Abdul Rafy

- The Next level table will be made which will contain all the binary of paired terms with elimination of one variable denoted by 'X'. (See table 4-11)

	Prime Implicants
0	1010
1	00x1
2	0x01
3	11x1
4	x10x
5	x10x

TABLE

Groups	Term	Index	TABLE	NEXT LEVEL Index	TABLE
1	$(m_4, m_5, m_{12}, m_{13})$	0	x10x		
	$(m_4, m_5, m_{12}, m_{13})$	1	x10x		

Checking Duplicates = Duplicates Found

After Removing Duplicates:

	Prime Implicants
0	1010
1	00x1
2	x10x
3	11x1
4	x10x

- As the Next Level Table has no terms or no two indexes are checked/marked, so **loop** will **stop** after this iteration.
  - Step 1 has been completed here……..

Step2 Starts after the Loop has been stopped

Manually the step 2 is performed in the table as follows:

TABLE 4–13

Prime Implicants		Minterms m_1	m_3	m_4	m_5	m_{10}	m_{12}	m_{13}	m_{15}
$B\overline{C}$ $(m_4, m_5, m_{12}, m_{13})$	x10x			✓	✓		✓	✓	
$\overline{A}\overline{B}D$ (m_1, m_3)	00x1	✓	✓						
$\overline{A}\overline{C}D$ (m_1, m_5)	0x01	✓			✓				
ABD (m_{13}, m_{15})	11x1							✓	✓
$A\overline{B}C\overline{D}$ (m_{10})	**1010**					✓			

Actually,

- In step 2, we determine all the minterms covered by each Prime Implicants.

  To achieve this we will compare our initial 'Table' and 'Prime Implicants' Table to find terms covered by each prime implicants.

Index	Minterm	Table
0	m_1	0001
1	m_4	0100
2	m_3	0011
3	m_5	0101
4	m_{10}	1010
5	m_{12}	1100
6	m_{13}	1101
7	m_{15}	1111

	Prime Implicants
0	1010
1	00x1
2	0x01
3	11x1
4	x10x

Indexes.e.g 4 means m10
4
0 2
0 3
5 7
1 3 5 6

Abdul Rafy

- Then we remove those Prime Implicants without which all minterms get covered. After that, we get the Essential Prime Implicants.

  To achieve this step, we will **one by one** store the indexes in one array without terms of one index or terms covered by any one implicants to see which prime implicant is unessential i.e. without which, we still have indexes **0** to **7** or **8** terms. We will store each index only once in the array. Unessential implicants will be removed if found. (2d shrink)

	First Iteration
Index	Without Index 0
0 (1010)	4
1 (00x1)	0 2
2 (0x01)	0 3
3 (11x1)	5 7
4 (x10x)	1 3 5 6
Array =	0 2 3 5 7 1 6
	Condition: 7 ≠ 8 (F) Index 0 is essential

Point Index 1

	Second Iteration
Index	Without Index 1
0 (1010)	4
1 (00x1)	0 2
2 (0x01)	0 3
3 (11x1)	5 7
4 (x10x)	1 3 5 6
Array =	4 0 3 5 7 1 6
	Condition: 7 ≠ 8 (F) Implicant 1 is Essen

Point Index 2

Remove 2	Third Iteration
Index	Without Index 2
0 (1010)	4
1 (00x1)	0 2
2 (0x01)	0 3
3 (11x1)	5 7
4 (x10x)	1 3 5 6
Array =	4 0 2 5 7 1 3 6
	Condition: 8 = 8 (T) Index 2 is UnEssen

Removing Unessential Term in Three Steps

(a)

Removing and Setting for next iteration	
Removed 2	Third Iteration
Index	Without Index
0 (1010)	4
1 (00x1)	0 2
3 (11x1)	5 7
4 (x10x)	1 3 5 6
Array =	4 0 2 5 7 1 3 6
	Condition: 8 = 8 (T) Index 2 is UnEssen

(b)

Removing and Setting for next iteration	
Shift Rows	Third Iteration
Index	Without Index
0 (1010)	4
1 (00x1)	0 2
2 (11x1)	5 7
3 (x10x)	1 3 5 6
Array =	4 0 2 5 7 1 3 6
	Condition: 8 = 8 (T) Index 2 is UnEssen

(c)

Removing and Setting for next iteration	
Point to 1	Third Iteration
Index	Without Index 1
0 (1010)	4
1 (00x1)	0 2
2 (11x1)	5 7
3 (x10x)	1 3 5 6
Array =	4 0 2 5 7 1 3 6
	Condition: 8 = 8 (T) Index 2 is UnEssen

Point Index 2

	Forth Iteration
Index	Without Index 2
0 (1010)	4
1 (00x1)	0 2
2 (11x1)	5 7
3 (x10x)	1 3 5 6
Array =	4 0 2 1 3 5 6
	Condition 7 ≠ 8 (F)

Point Index 3

	Fifth Iteration
Index	Without Index 3
0 (1010)	4
1 (00x1)	0 2
2 (11x1)	5 7
3 (x10x)	1 3 5 6
Array =	4 0 2 5 7
	Condition 5 ≠ 8 (F)

Abdul Rafy

- If total no. of terms covered by these Essential Prime Implicants are equal to the no. of min terms taken from the user, these prime implicants are our Final Expression.

	Prime Implicants
0	1010
1	00x1
3	11x1
4	x10x

Final Expression = $A_1B_1'C_1D_1' + A_1'B_1'D_1 + A_1B_1D_1 + B_1C_1'$

Step 2 Completed…….

Program: Write a Program that inputs no. of variables for Binary length, total number of minterms and each minterm one by one, and finds the simplified Expression based.

(OR)

Program: Write a program that uses Quine Mc Cluskey method to simplify expression on given minterms.

Program: Write a program that demonstrate K-Map and Quine Mc Cluskey method and output simplified expression.

//It is orignal copy so you may delete what code is commented.I have not deleted because it is orignal code and what code is comment can also be usedful. In other copies, it is cleaned.

```
#include<iostream>

#include<fstream> // Used in exporting results

using namespace std;

void display2d(int** oldarray,int rows,int cols) {

	for(int i=0; i<rows; i++) {

		for(int j=0; j<cols; j++) {

			cout<<oldarray[i][j];

		}

		cout<<endl;

	}

}

void input_minterms(int& size, int*& minterms, int& length) {

	cout << "Enter the number of variables:";

	cin >> length;

	cout << "Enter the number of minterms:";

	cin >> size;

	minterms = new int[size];

	for (int i = 0; i < size; i++) {

		cout << "Enter Min No. " << i + 1 << ":";

		cin >> minterms[i];

	}

}//This function is taking inputs: 'variables','total no. of minterms',and each 'minterms'  from the user

//OR you may use the following input_minterms function if you donot know the number of terms, but before using it define the regrow function before this function, because it will be using that(regrow) function

/*void input_minterms(int& size, int*& minterms, int& length) {

	int num=0;
```

Abdul Rafy

```
	cout << "Enter the number of variables:";
	cin >> length;
	while(num!=-1){
		cout << "Enter Min No. " << size + 1 << ":";
		cin >> num;
		if(num!=-1)
		{
			regrow(minterms,size,num);
			size++;
		}
	}
}*/
int* findbinary(int length, int minterms) {
	int* binary = new int[length];
	for (int i = 0; i < length; i++) {
		binary[(length - 1) - i] = minterms % 2;
		minterms/=2;
	}
		return binary;
}// This function is finding binary corresponding to given input.
int checkones(int row, int length, int** table) {
	int countones = 0;
	for (int j = 0; j < length; j++) {
		if (table[row][j] == 1) {
			countones++;
		}
	}
	return countones;
}//This function will help to make comparison between terms of adjacent groups.
void display(int size,int* cols,int** pairs,int* minterms)
{
	for(int i=0;i<size;i++){
```

Abdul Rafy

```
		cout << "Pairs with cols m" << i << " are";
		for (int j = 0; j < cols[i]; j++) {
			cout << pairs[i][j] << ",";
		}
		cout << endl;
	}
}//This function is displaying the pairs formed by each minterm.
void strcpy(int* oldarray, int* newarray, int col) {
	for (int i = 0; i < col; i++) {
		newarray[i] = oldarray[i];
	}
}
void regrow(int*& oldarray, int col, int value) {
	int* newarray=NULL;
	newarray = new int[col+1];
	strcpy(oldarray, newarray, col);
	newarray[col] = value;
	delete[] oldarray;
	oldarray = newarray;
}
void compare(int size, int length, int** table,int**& pairs,int *minterms,int*&cols) {
	cout<<endl<<"All Possible Pairs"<<endl;
	int check1 = 0, check2 = 0,*oldarray=NULL;
	cols = new int[size] {};
	pairs = new int* [size];
	for (int k = 0; k < size; k++) {
		check1 = checkones(k, length, table);
		for (int i = 0; i < size; i++) {
			check2 = checkones(i, length, table);
			if (check1 == check2 - 1) {
				regrow(oldarray,cols[k],i);
				cols[k]++;
```

Abdul Rafy

```
                }
                check2++;
            }
            pairs[k] = oldarray;
            oldarray = NULL;
            check1++;
        }
        display(size, cols, pairs,minterms);
}
void strcpy(int* newarray,int*oldarray,int size,int value) {
        int l=0;
        for(int i=0; i<size; i++) {
            if(oldarray[i]!=value) {
                newarray[l]=oldarray[i];
                l++;
            }
        }
}
void shrink(int*&oldarray,int& size,int value) {
        int *newarray=NULL;
        newarray=new int[size-1];
        strcpy(newarray,oldarray,size,value);
        size--;
        delete[] oldarray;
        oldarray=newarray;
}
void requiredpairs(int size, int length, int** table,int** pairs,int *minterms,int *cols) {
        int count=0;
        for(int i=0; i<size; i++) {
            for(int j=0; j<cols[i]; j++) {
                for(int k=0; k<length; k++) {
                    if(table[i][k]==table[pairs[i][j]][k]) {
```

Abdul Rafy

```
                                        count++;
                                }
                        }
                        if(!(count== length-1)) {


                                shrink(pairs[i],cols[i],pairs[i][j]);//here pairs[i] will act as our old pointer, cols will act as our size,and pairs[i][j] will act as our value, we will find the index of that value and delete it.
                                j--;//because when we remove an element in the middle, the next coming element will automatically come behind without any need of incrementing j so we should stop j from being j++ and should remain at the same position becasuse on shrinking all elements shits towards left, therefore we are doing j-- to cancel the effect of j++.
                        }
                        count=0;
                }
        }
        cout<<endl<<"Required Pairs"<<endl;
        display(size, cols, pairs,minterms);
}
int findrows(int** pairs,int*cols,int size)
{
        int count=0;
        for(int i=0;i<size;i++)
        {
                for(int j=0;j<cols[i];j++)
                {
                        count++;
                }
        }
        return count;
}
void eliminating(int size,int length,int** table,int** pairs,int* cols,int**&table1,int& size1)
{
        int l=0;//l is being used to increment values
```

Abdul Rafy

```
	//count all pairs to get the  number of  rows which will be stored in size1 variable
	size1=findrows(pairs,cols,size);
	table1=new int*[size1];
	for(int i=0;i<size1;i++)
	{
		table1[i]=new int[length];
	}
	for(int i=0;i<size;i++)
	{
		for(int j=0;j<cols[i];j++)
		{
			for(int k=0;k<length;k++)
			{
				if(table[i][k]==table[pairs[i][j]][k])
				{
					table1[l][k]=table[pairs[i][j]][k];
				}
				else
					table1[l][k]=-1;
			}
			l++;
		}
	}
	cout<<endl;
	display2d(table1,size1,length);
	cout<<endl;
}
void method1(int*& implicants1,int& total1,int size,int* cols)
{
	 cout<<"Method 1"<<endl;
	bool flag=false;
	for(int i=0;i<size;i++)
```

Abdul Rafy

```
	{
		if(cols[i]==0)
		{
			cout<<"value="<< cols[i]<<endl;
			flag=true;
		}
		if(flag)
		{
			cout<<"Term: "<<i<<endl;
			regrow(implicants1,total1,i);
			total1++;
		}
		flag=false;
	}
}
void method2(int*& implicants2,int& total2,int** pairs,int size,int* cols)
{
	cout<<"Method 2"<<endl;
	bool flag=false;
	for(int k=0;k<size;k++)//k is to match all min terms have used or not.
	{
	 for(int i=0;i<size;i++)
		{
			for(int j=0;j<cols[i];j++)
			{
				if(k==pairs[i][j])
				{
					cout<<k<<"=="<<pairs[i][j]<<endl;
					flag=true;
					break;
				}
				else
```

Abdul Rafy

```
                                        flag=false;
                        }
                        if(flag)
                        {
                                break;
                        }
                }
                if(!flag)
                {
                        cout<<"Term: "<<k<<endl;
                        regrow(implicants2,total2,k);
                        total2++;
                }
                flag=false;
        }
}
void common(int*& implicants,int* implicants1,int* implicants2,int& total,int total1,int total2)
{
        cout<<"Intersection or UnMarked term"<<endl;
        for(int i=0;i<total1;i++)
        {
                for(int j=0;j<total2;j++)
                {
                        if(implicants1[i]==implicants2[j])//The equality no doubt is the result of XOR not of AND but the
sense used to combine two function to get result is the intersection of two sets which is AND in propositional logic.
                        {
                                regrow(implicants,total,implicants1[i]);//or you may use
regrow(implicants,total,implicants2[j]); becuase this value is common in both
                                total++;
                        }
                }
        }
}
```

Abdul Rafy

```
void strcpy2d(int**oldarray,int**newarray,int rows,int cols) {
	for(int i=0; i<rows; i++) {
		newarray[i]=new int[cols];
		for(int j=0; j<cols; j++) {
			newarray[i][j]=oldarray[i][j];
		}
	}
}
void addnewcolumn(int** newarray,int*newcolumn,int rows,int cols) {
	newarray[rows]=new int[cols];
	for(int j=0; j<cols; j++) {
		newarray[rows][j]=	newcolumn[j];
	}
}
void regrow2d(int**&oldarray,int rows,int cols,int*newcolumn) {
	int**newarray;
	newarray=new int*[rows+1];
	strcpy2d(oldarray,newarray,rows,cols);
	addnewcolumn(newarray,	newcolumn,rows,cols);
	for(int i=0; i<rows; i++) {
		delete[] oldarray[i];
	}
	delete[] oldarray;
	oldarray=newarray;
}
void strcpyshrink(int** oldarray, int** newarray, int row, int column,int removerowindex)
{
	int l = 0;
	for (int i = 0;i < row;i++)
	{
		if (i != removerowindex) {
			newarray[l] = new int[column];
```

Abdul Rafy

```
                for (int j = 0;j < column;j++)
                {
                        newarray[l][j] = oldarray[i][j];
                }
                l++;
            }
        }
}
void shrink2d(int**& oldarray, int row, int column,int removerowindex)
{
        int** newarray=nullptr;
        newarray = new int*[row-1];
        strcpyshrink(oldarray, newarray, row, column,removerowindex);
        for (int i = 0;i < row;i++)
        {
            delete[]oldarray[i];
        }
        delete[]oldarray;
        oldarray = newarray;
}
void RemovingDuplication(int**& primeimplicants,int& row,int length)
{//the reason of using & with row and primeimplicants is that i'm shrinking and changing the allocation place to which the point is pointing.
        bool flag=false;
        int count=0;
        for(int k=0;k<row;k++)
        {
            for(int i=0;i<row;i++)
            {
                for(int j=0;j<length;j++)
                {
                    if(primeimplicants[k][j]==primeimplicants[i][j])
```

Abdul Rafy

```
                        {
                                flag=true;
                        }
                        else
                        {
                                flag=false;
                                break;
                        }
                }
                if(flag)
                {
                        count++;
                }
                        if(flag&&count>1)
                {
                        cout<<"implicants "<<i<<" are duplicates"<<endl;
                        shrink2d(primeimplicants, row, length,i);
                        row--;
                        i--;//becausenext row has been shifted in the place which is removed therefore we
donot to increment this time
                }
            }
//OR          if(flag&&count>1) but above one is better, you may delete what is here in the comment
//              {
//                      cout<<"implicants "<<k<<" are duplicates"<<endl;
//                      shrink2d(primeimplicants, row, length,k);
//                      row--;
//                      k--;//becausenext row has been shifted in the place which is removed therefore we
donot to increment this time
//              }
            count=0;
        }
}
```

Abdul Rafy

```
void CheckingTermsUsed(int size,int length,int** table,int** pairs,int* cols,int**& primeimplicants,int& row)
{
	/*
	way 1
	-----------------------------------------------------------------------------------------------------------------------------------
	The following two steps will be taken place to check which term is,remaining and has not take part in any pair, or you may say, prime implicant
	Test data number of variables= 4, total minterms=8, minterms=1,3,4,5,10,12,13,15
	1- I will see for which row the col[i] is zero if it is zero
	  it may mean that this minterm or row number may have not taken place in any pairing.
	for example if i use the test data, then I see that by using first method I get the terms: 1,4,7, which implies 3,10,15 ,
	however actual result is 4 only, which implies 10.
	2-  The min terms(0-size) will be compared with all terms in the pairs, if these terms matces with any of the term
	in "pairs" 2d array, it means they have come in pairs, and those terms which donot match with any terms in 2darray"pairs",
	it means that these are the our prime implicants and has not take part in any pairing.
	for example. if i use the test data when i use 2nd method,then I see that by using second method I get the terms: 0,2,4, which implies 1,4,10. So, if
	I use AND, I will get the result common in both which is term 4 or term 10. For more clarification you can see image with quine mccluskey
	way2
	Way 1 takes more variables
	-----------------------------------------------------------------------------------------------------------------------------------
	To get the actual result we can also use these two steps
	1- Pass(input) the result, acquired from step 1, to step 2.
	2-The terms come from 1st method whould be compared with the terms in 2darray'pairs' those, which would match, will not remove from implicants and those,which will not match, will remain in impliant array. But the program will have to be changed for way 2
	because the first way is being used here in which boolean operation AND is used. But you can also use way2.
	Way 2 takes less variables
	-----------------------------------------------------------------------------------------------------------------------------------
	*/
	// i donot want to use the word "size" again because i have already used it for total no of given terms
```

Abdul Rafy

```
	int *implicants1=nullptr,total1=0;
	int *implicants2=nullptr,total2=0;
	int *implicants=nullptr,total=0;//to store actual results
	method1(implicants1, total1,size,cols);
	method2(implicants2,total2,pairs,size,cols);
	common(implicants,implicants1,implicants2,total,total1,total2);//here intersection operation of set which is called AND, in boolen language, will be applied. The "implicant" array will store the common term in both "implicant1" and "implicant2" and "total" will store the actual no of terms not being used in any pairing
	for(int i=0;i<total;i++)
	{
		regrow2d(primeimplicants,row,length,table[implicants[i]]);
		row++;
	}
	cout<<"Before checking Duplicates"<<endl;
	display2d(primeimplicants,row,length);
	cout<<"After checking Duplicates"<<endl;
	RemovingDuplication(primeimplicants,row,length);
	display2d(primeimplicants,row,length);
	cout<<"Prime Implicant from this Level Table:";
	for(int i=0;i<total;i++)
	{
		cout<<implicants[i]<<",";
	}
	cout<<endl<<"Total Prime Implicants Binary"<<endl;
	display2d(primeimplicants,row,length);
	cout<<endl;
}
void delete2d(int **pointer,int rows)
{
	for(int i=0;i<rows;i++)
	{
		delete[]pointer[i];
	}
```

Abdul Rafy

```
	delete[]pointer;
}
void copy(int** table1,int size1,int length,int**& table)
{
	table=new int*[size1];
	for(int i=0;i<size1;i++)
	{
		table[i]=new int[length];
		for(int j=0;j<length;j++)
		{
			table[i][j]=table1[i][j];
		}
	}
}
bool stopingloop(int size1)
{
	if(size1==0)
	{
		return false;
	}
	else
		return true;
}
int** MintermsBinaryTable(int size,int length,int* minterms)
{
	int **table;
	table=new int*[size];
	for (int i = 0; i < size; i++) {
		table[i] = findbinary(length, minterms[i]);
	}
	return table;
}
```

Abdul Rafy

```
void diplay2ddifferentcolumn(int** termscovering,int** primeimplicants,int row,int* columns,int length)
{
	for(int i=0;i<row;i++)
	{
		for(int j=0;j<length;j++)
		{
			cout<<primeimplicants[i][j];
		}
		cout<<" is covering the terms: ";
		for(int j=0;j<columns[i];j++)
		{
			cout<<"m"<<termscovering[i][j]<<",";
		}
			cout<<endl;
	}
	cout<<endl;
}
void findingTermsCoveredByEactPrimeImplicant(int**& termscovering,int** primeimplicants,int row,int length,int**
table,int MintermsTableRows,int*& columns)
{
	bool flag=false;
	for(int i=0;i<row;i++)
	{
		cout<<"new"<<endl;
		for(int j=0;j<MintermsTableRows;j++)
		{
			for(int k=0;k<length;k++)
			{
				if( (primeimplicants[i][k]==table[j][k]) || (primeimplicants[i][k]==-1) )
				{
					cout<<primeimplicants[i][k]<<" == "<<table[j][k]<< "||"<<
primeimplicants[i][k]<<"=="<<-1<<endl;
					flag=true;
```

Abdul Rafy

```
                }
                else
                {
                        flag=false;
                        break;
                }
        }
        cout<<endl;
        if(flag)
        {
                cout<<"Term: "<<j<<" in "<<i<<endl;
                regrow(termscovering[i],columns[i],j);//j is the term 0 to size, which relative actual
terms number is stored in 'minterms' 1d array
                //termscovering[i] is refering to something garbage which you're going to send by call
by reference and may not accept or swallow by the pointer in the dummy parameter of the defined function
                columns[i]++;
        }
    }
}
//displaying the terms corresponding to each prime Implicants
diplay2ddifferentcolumn(termscovering,primeimplicants,row,columns,length);
}
bool checkingduplicate(int* array,int lengthOfArray,int termscovering)
{
    bool flag;
    for(int i=0;i<lengthOfArray;i++)
    {
        if(array[i]!=termscovering)
        {
            flag=true;
        }
        else
        {
```

Abdul Rafy

```
                flag=false;
                break;
            }
        }
        return flag;
}
void SeparatingEssentialfromNonEssentialPrimeImplicants(int**& primeimplicants,int& row,int length,int
MintermsTableRows,int** termscovering,int* columns)
{
        cout<<"This is mintable rows:"<<MintermsTableRows<<endl;
        int lengthOfArray=0,*arr=nullptr; //array should store without duplicates terms from 0 to so on
        bool flag=true;
        for(int i=0;i<row;i++)
        {
            cout<<"Terms covering without involving prime implicant at index "<<i<<endl;
            flag=true;
            for(int j=0;j<row;j++)
            {
                if(i!=j)
                {
                    for(int k=0;k<columns[j];k++)
                    {
                        cout<<termscovering[j][k]<<" ";
                        if(lengthOfArray!=0)
                        {
                            flag=checkingduplicate(arr,lengthOfArray,termscovering[j][k]);// this is
to store each term without any duplicate
                        }
                        if(flag)
                        {
                            regrow(arr,lengthOfArray,termscovering[j][k]);
                            lengthOfArray++;
                        }
```

Abdul Rafy

```
                        }

                }
        }
        cout<<endl;
        cout<<"Without Duplication:"<<endl;
        for(int i=0;i<lengthOfArray;i++)
        {
                cout<<arr[i]<<",";
        }
        cout<<endl;
        if(lengthOfArray==MintermsTableRows)
        {
                shrink2d(primeimplicants,row,length,i);
                row--;
                i--;
                cout<<lengthOfArray<<"=="<<MintermsTableRows<<endl;
                diplay2ddifferentcolumn(termscovering,primeimplicants,row,columns,length);
        }
        delete[] arr;
        arr=nullptr;
        lengthOfArray=0;
    }
}
int main() {
    //length will store the number of variables. Size is the total number of minterms
    //table is storing binary. pairs variable is storing pairs. cols is the columns of 2d array pairs.
    // prime implicants are still storing all those terms which are unchecked.
    // row here are the number of rows for 2d array "primeimplicants", where 'length' is its number of columns
    //size1 is the number of rows for table1 storing immediate results like -1101 or 0-1-11, where 'length' is its number of columns
    int* minterms=NULL, ** table=NULL,MintermsTableRows=0,**pairs=NULL, size=0, length=0,*cols=NULL,**primeimplicants=nullptr,row=0,**table1=nullptr,size1=0;
```

Abdul Rafy

```
bool flag=1; //or bool flag=true;
input_minterms(size, minterms, length);
table=MintermsBinaryTable(size,length,minterms);
MintermsTableRows=size;//the purpose of storing size in this varibable is that, we need it at last and till last size changes many time.
/*
The Queen McCluskey Method consits of two steps
step 1 of Queen McCluskey Method
Comparing terms and Determining Prime implicants
*/
//-------------------------------------------------------------------------------------
while(flag){
cout<<endl<<"Truth Table"<<endl;
display2d(table, size,length);// Displaying truth values of minterms
compare(size, length, table,pairs,minterms,cols);
requiredpairs(size, length, table,pairs,minterms,cols);
cout<<endl<<"First Level";
//process
eliminating(size, length, table,pairs,cols,table1,size1);// i am also making next level table in this function
//copy
CheckingTermsUsed(size, length,table,pairs,cols,primeimplicants,row);//checking whether all terms has come in used and finding which term is not stored to store its boolean expression for final expression
flag=stopingloop(size1);//loop should stop when there is nothing more in first level table and it happens when size1=0,meaning no minterm have been marked, so no more group is made or will be made,nothing in first level table.Hence the loop should stop if x=1; loop should run, if x=0 loop should stop
delete2d(table,size);
table=nullptr;
copy(table1,size1,length,table);
delete2d(table1,size1);
table1=nullptr;
delete2d(pairs,size);
size=size1;//because value in 'size' is needed to deallocate 'pairs'
size1=0;
```

Abdul Rafy

```
        }
/*-----------------------------------------------------------------------
Step2: Separating Essential Prime Implicants and Finishing Non Essential Prime Implicants
-----------------------------------------------------------------------------*/
//At the end of step 1, table1 has nothing which is copying to table so table will also has nothing but it will be refering a memory address given by table1 having size 0.By the way. table1 is referring to none and has size1=0, i may utilize it but i donot want to burden on memory and want to use table
        delete table;
        table=nullptr;
        int**termscovering=nullptr,* columns=nullptr;//the 1d array 'columns' will be used for storing total no. of minterms which can be different, becuase different prime implicant may cover different number of minterms
        termscovering=new int*[row];//In terms covering the terms will be stored corresponding to primeimplicants which are covered by each prime implicants
        for(int i=0;i<row;i++)
        {
                termscovering[i]=nullptr;
        }
        columns=new int[row]{};
        table=MintermsBinaryTable(MintermsTableRows,length,minterms);
        cout<<"Minterms Truth Table"<<endl;
        display2d(table, MintermsTableRows,length);// Displaying truth values of minterms
        findingTermsCoveredByEactPrimeImplicant(termscovering,primeimplicants,row,length,table,MintermsTableRows,columns);
        SeparatingEssentialfromNonEssentialPrimeImplicants(primeimplicants,row,length,MintermsTableRows,termscovering,columns);//The idea used here is that those terms without which all minterms 0 to size get covered are our minterms.


//-------------------------------------------------------------


        ofstream file;
        file.open("result.txt");
        if(file)
        {
         cout<<"Final Expression"<<endl<<"= ";
```

Abdul Rafy

```
        for(int i=0;i<row;)
        {
                for(int j=0;j<length;j++)
                {
                        if(primeimplicants[i][j]!=-1)
                        {
                                cout<<"A"<<(length-1)-j;
                                file<<"A"<<(length-1)-j;
                        }
                        if(primeimplicants[i][j]==0)
                        {
                                cout<<"'";
                                file<<"'";
                        }
                }
                i++;
                if(i<row){
                cout<<" + ";
                file<<" + ";
                }
        }
        }//-------------------------------------------------------------
}
```

Abdul Rafy